
House Price Prediction

Jeremy Guan Cheng-Cheng Liu Xudong Zhang Yixiao Li

Abstract

This project investigates house price prediction using the Kaggle House Prices dataset, with a focus on comparing linear, nonlinear, and ensemble-based machine learning models. We evaluated Linear Regression, Ridge, Lasso, Gradient Boosting, and XGBoost, together with hybrid and stacking approaches. Our preprocessing pipeline included missing value handling, categorical feature encoding, and feature transformation. We found that purely linear models were insufficient for capturing complex relationships in housing data, while nonlinear ensemble methods significantly improved prediction performance. Furthermore, combining linear and nonlinear models through hybrid averaging and stacking produced more stable and accurate predictions than individual models. Our results highlight the importance of feature engineering and ensemble learning in structured regression tasks.

1. Introduction

This project investigates house price prediction using the Kaggle House Prices: Advanced Regression Techniques dataset [Kaggle House Prices Competition](#). Because housing prices depend on complex feature interactions and nonlinear relationships, this dataset provides an interesting setting for studying the performance and limitations of different machine learning models.

Our project began as a comparison of regression approaches including Linear Regression, Ridge, Lasso, and nonlinear boosting methods. Over time, the project evolved into a more analytical investigation of when linear models become insufficient and whether combining linear and nonlinear models can improve the prediction's performance. We explored ensemble approaches including weighted hybrid models and stacking architectures.

When reviewing prior work and public Kaggle submissions for this competition, we found that top-performing solutions relied heavily on feature engineering and ensemble learning techniques. The solution by [House Prices Solution \[top 1\]](#) ([jesucristo, 2017](#)) demonstrated the effectiveness of combining multiple regression and boosting models for

improved performance. We also found [House Prices EDA](#), [Lasso](#), [LightGBM](#) ([mviola, 2018](#)) particularly useful for combining linear regularized models such as Lasso with nonlinear boosting approaches such as LightGBM. These prior solutions helped with our feature engineering process.

Inspired by these approaches, we investigate whether combining linear and nonlinear models can consistently outperform individual models alone. Our work focuses on predictive accuracy, and the effectiveness of ensemble learning in structured regression tasks.

2. Methods

Figure 1 presents the overall machine learning workflow used in this project. Our methodology consists of four main stages: data preprocessing, linear regression baselines, nonlinear ensemble models, and ensemble combination strategies.

2.1. Data Preprocessing

The Kaggle House Prices dataset contains both numerical and categorical features with substantial missing values. Missing categorical values representing the absence of a property feature (e.g., pool or fence) were encoded accordingly, while numerical missing values were imputed using the median. Categorical variables were transformed using one-hot encoding.

To improve model stability and reduce skewness, logarithmic and power transformations were applied to selected variables. In particular, the target variable `SalePrice` exhibited a strongly right-skewed distribution, motivating the use of a logarithmic transformation to reduce the influence of outliers and improve distribution symmetry.

2.2. Linear Regression Models

We first established baseline performance using ordinary least squares linear regression. These models served as baselines for analyzing the limitations of linear assumptions in housing price prediction. To improve generalization and reduce overfitting, we additionally evaluated Ridge and Lasso regression models. Ridge regression introduces an L2 regularization penalty, while Lasso regression applies an L1 penalty that performs implicit feature selection. Regu-

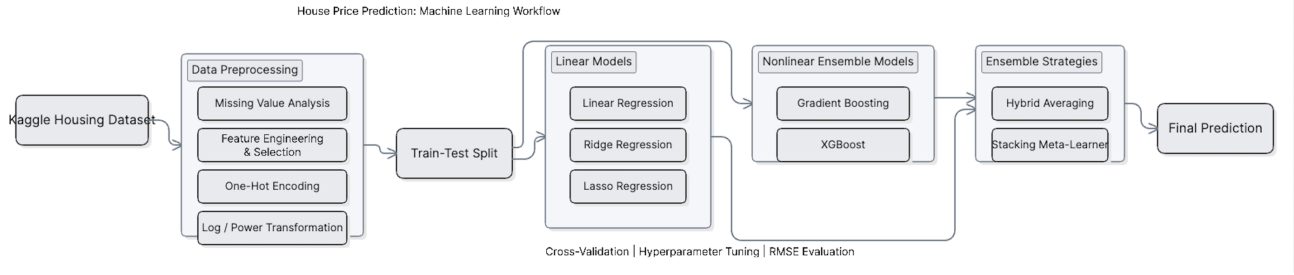


Figure 1. Overview of the machine learning workflow used for house price prediction.

larization strengths were selected using cross-validation to balance model complexity and performance.

2.3. Nonlinear Ensemble Models

To capture nonlinear relationships and complex feature interactions within the housing dataset, we further evaluated nonlinear ensemble methods including Gradient Boosting Regression and XGBoost. Unlike linear models, boosting methods iteratively train decision trees on residual errors from previous predictors, enabling the models to learn nonlinear structures in the data.

We additionally conducted hyperparameter sensitivity analysis using cross-validation together with grid search and randomized search procedures. Hyperparameters including learning rate, tree depth, number of estimators, subsampling ratio, and minimum leaf size were tuned to improve predictive performance and reduce overfitting.

2.4. Ensemble Combination Strategies

To systematically study whether combining linear and nonlinear models improves prediction performance, we implemented both weighted hybrid averaging and stacking architectures.

Our weighted hybrid model combines predictions from Ridge regression and Gradient Boosting through a convex combination:

$$\hat{y}_{\text{hybrid}} = \alpha \hat{y}_{\text{ridge}} + (1 - \alpha) \hat{y}_{\text{GB}}$$

where $\alpha \in [0, 1]$ controls the relative contribution of the linear Ridge regression model and the nonlinear Gradient Boosting model.

We further implemented stacking architectures in which predictions from multiple base learners were used as inputs to a meta-learner that generated the final prediction. Multiple ensemble configurations were evaluated, including combinations of regularized linear models and nonlinear boosting models. These ensemble approaches aim to leverage the strengths of different model families while reducing the

weaknesses of individual predictors.

3. Results

We evaluated multiple machine learning models for house price prediction using the Kaggle House Prices dataset, which contains 1460 training examples and 79 explanatory variables describing residential homes in Ames, Iowa. All models were trained using 5-fold cross-validation and evaluated using Root Mean Squared Error (RMSE) in log-space.

3.1. Model RMSE Comparison

Figure 2 summarizes the final RMSE performance across all implemented models.

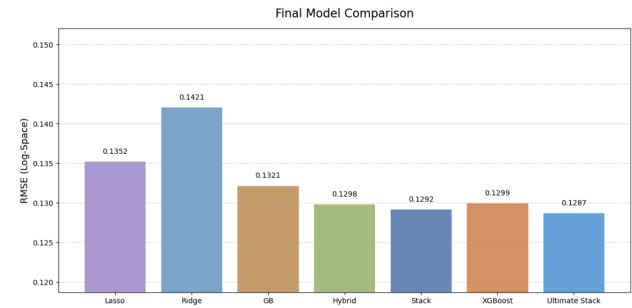


Figure 2. Final RMSE comparison across all models. Lower RMSE indicates better performance.

Among the linear models, Lasso Regression (Tibshirani, 1996) achieved slightly lower RMSE than Ridge Regression (Hoerl & Kennard, 1970) because L1 regularization performs implicit feature selection by shrinking less important coefficients to zero.

Nonlinear models performed substantially better overall. Gradient Boosting (Friedman, 2001) significantly outperformed the linear approaches, suggesting that housing prices contain strong nonlinear relationships that simple linear models cannot fully capture.

The Hybrid and Stacking ensemble methods (Wolpert, 1992; Breiman, 1996) provided additional improvements beyond standalone Gradient Boosting. XGBoost (Chen & Guestrin, 2016) achieved performance comparable to Gradient Boosting, while the Ultimate Stacking model achieved the best overall RMSE of 0.1287 by combining predictions from Lasso Regression, Gradient Boosting, and XGBoost.

Although the improvements from ensemble methods were relatively modest, the results suggest that combining more diverse nonlinear learners provides additional complementary predictive information.

3.2. Ultimate Stack Model

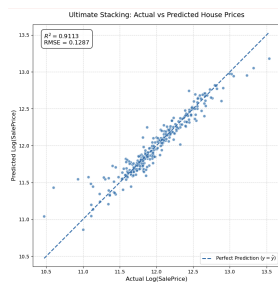


Figure 3. Ultimate Stacking predictions versus actual house prices in log-space. The dashed diagonal line represents perfect predictions.

Since the Ultimate Stacking model achieved the lowest overall RMSE among all evaluated approaches, we focus our detailed prediction analysis on this model. Presenting only the best-performing model provides a clearer visualization of overall prediction quality and remaining errors without introducing redundant plots for weaker models.

Figure 3 shows the prediction quality of the Ultimate Stacking model on the final test set. Most predictions lie close to the diagonal reference line, indicating a strong relationship between predicted and actual housing prices. The model achieved an R^2 score of 0.9113, indicating that the ensemble model successfully explained most of the variance in house prices.

Although most homes were predicted accurately, several outliers remain visible, particularly among extremely expensive homes. These prediction errors likely occur because luxury properties and rare neighborhood characteristics are underrepresented in the training data. In addition, some housing attributes may interact non-linearly in ways that are difficult for tabular features to fully capture. Overall, the prediction results demonstrate that ensemble methods generalized effectively and produced highly accurate predictions across a wide range of housing prices.

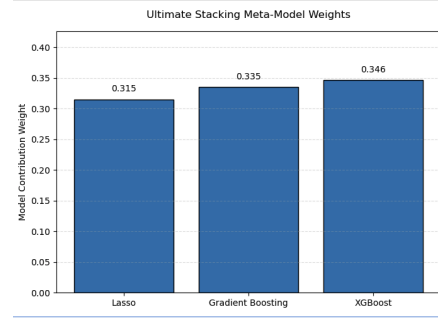


Figure 4. Meta-model weights learned by the Ultimate Stacking ensemble. Larger weights indicate stronger contribution to the final prediction.

3.3. Ultimate Stack Weights

Figure 4 shows the contribution weights learned by the stacking meta-model. XGBoost and Gradient Boosting received the largest weights, indicating that nonlinear tree-based learners contributed most strongly to the final predictions. Lasso Regression still provided complementary predictive information, suggesting that combining both linear and nonlinear modeling approaches improved overall ensemble performance.

4. Process

In this section, we will discuss our works, explorations, and iterations that led to the final solution.

4.1. Data Pre-processing: From Naive Imputation to Stratified Strategy

Initially, we implemented a naive imputation strategy by replacing all missing values with the feature mean. However, iterative diagnostics revealed that this approach violated the underlying statistical logic, as extreme outliers frequently pulled the mean away from the true central tendency.

To resolve this, we transitioned to a skewness-based imputation mechanism: features with an absolute skewness greater than 1.0 were imputed with the median, while roughly symmetric features retained mean imputation. For categorical data, we moved beyond uniform mode imputation by incorporating domain-specific insights. We observed that high missing rates often signaled the physical absence of a facility (e.g., "No Basement") rather than random omission. Consequently, we pivoted to imputing "None" for facility-related features and reserved mode imputation only for specific features like `Electrical`, where missingness was diagnosed as a random human error due to its extremely low frequency and high class dominance.

4.2. Model Evolution: Overcoming Baseline Pathologies

Our modeling process was highly iterative, driven by diagnosing and resolving successive algorithmic pathologies. We established our initial baseline using unregularized Ordinary Least Squares (OLS) regression. Instead of standard errors, we observed a catastrophic failure in predictive stability, yielding a sub-optimal R^2 score of 0.6797 and violent RMSE fluctuations across cross-validation folds.

By visualizing the coefficient weights, we discovered a severe case of algorithmic hijacking caused by highly collinear categorical variables. The OLS model created “predictive landmines” by assigning extreme negative weights (e.g., -3.6571) to sparse configurations. Lacking a penalty term, it suffered from perfect multicollinearity, arbitrarily distributing identical weights across related `_None` features.

Diagnosing this pathological behavior provided the definitive mathematical justification for our transition to regularized models (Lasso and Ridge). While L1 and L2 penalties successfully collapsed the inflated multicollinear weights, they introduced a new vulnerability: extreme sensitivity to feature scales and skewness, which we ultimately resolved using the Yeo-Johnson transformation.

However, even after stabilizing the linear baselines, we recognized a fundamental limitation: linear models cannot inherently capture complex, non-linear interactions between housing features (e.g., the compounding value of high overall quality in a premium neighborhood). This realization catalyzed our architectural pivot to the Gradient Boosting Regressor (GBR). By utilizing an ensemble of decision trees, GBR fundamentally bypassed the multicollinearity trap, required less rigid assumptions about data distribution, and successfully captured the non-linear nuances of the real estate market. Ultimately, this successful transition from linear frameworks to tree-based ensembles not only improved predictive stability but also laid a robust foundation for our subsequent deployment of advanced algorithms, including XGBoost and Stacking architectures.

4.3. Feature Transformation: Resolving Weight Distortion

Upon applying regularized linear models (Lasso and Ridge), we observed a stark discrepancy in feature importance compared to our Gradient Boosting Regressor (GBR). The unstandardized linear models disproportionately inflated the weights of one-hot encoded categorical features, such as `Neighborhood`. In contrast, GBR logically prioritized fundamental structural metrics like `OverallQual` and `GrLivArea`.

We diagnosed this as a scale-induced distortion: because one-hot features are bounded between 0 and 1, the linear models assigned them massive coefficients to capture their

impact on the target variable. To unify the scales, we initially applied standard scaling (`StandardScaler`). Surprisingly, this degraded the models’ predictive performance. Further investigation revealed that standard scaling failed to correct the underlying high skewness of our numerical features, which violated the homoscedasticity assumption of linear models.

To resolve both the scale disparity and the non-normality, we implemented a significant engineering pivot by introducing the **Yeo-Johnson power transformation**. This robust technique successfully normalized highly skewed features while standardizing their scales. Consequently, the predictive accuracy of our regularized models surpassed the unstandardized baseline, and their feature importance rankings perfectly aligned with the core structural drivers identified by the GBR model.

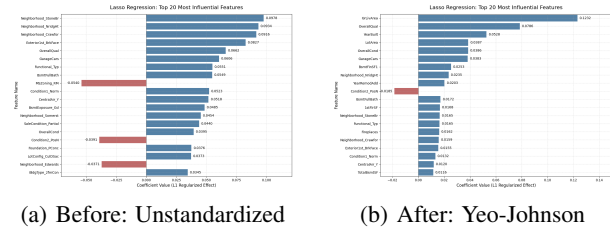


Figure 5. Comparison of Lasso feature importance before and after resolving scale distortion and non-normality. The transformed Lasso perfectly aligns with GBR’s feature priority.

Table 1. Impact of Feature Transformation on Model Performance (CV RMSE)

Strategy	Baseline	Lasso	Ridge
Unstandardized	0.1657	0.1445	0.1426
StandardScaler Only	0.1636	0.1480	0.1437
Yeo-Johnson Transform	0.1501	0.1323	0.1346

4.4. Model Interpretability: Diagnosing Counterintuitive Coefficients

Even after the Yeo-Johnson transformation optimized our regularized models, a granular inspection of the feature importance revealed counterintuitive predictive behaviors. For instance, the linear models assigned a negative weight to feature `Condition2_PosN` (proximity to a park or greenbelt), paradoxically implying that beneficial neighborhood amenities decrease home value.

As empirical evidence of this vulnerability, we traced this specific coefficient back to its raw training instances (Table 2). We discovered that this algorithmic anomaly was driven by a combination of categorical sparsity and isolated outliers. Datapoint Id 524 represents a massive anomaly: despite boasting the maximum `OverallQual` score of 10 and a sprawling `GrLivArea` of 4,676 sq.ft., its sale price is

severely depressed at \$184,750.

Table 2. Diagnosis of Outlier-Induced Weight Distortion

Id	SalePrice	Neighborhood	OverallQual	GrLivArea	Cond.1	Cond.2
524	\$184,750	Edwards	10	4,676	PosN	PosN
826	\$385,000	NridgHt	10	2,084	PosN	PosN

Because the PosN condition is extremely sparse within the dataset, the global linear model was forced to disproportionately punish this specific feature weight to mathematically accommodate the localized price collapse of this single outlier.

This empirical finding exposed a residual vulnerability in global linear models: their extreme susceptibility to localized noise within sparse features. This diagnosis served as the final catalyst for our transition to advanced tree-based ensembles. Unlike linear models, algorithms like the Gradient Boosting Regressor (GBR) and XGBoost can effectively isolate such anomalous datapoints into deep, specific splits, preventing localized noise from distorting the global feature logic.

4.5. Hyperparameter Optimization: From Grid to Stochastic Search

The performance of a Gradient Boosting Regressor (GBR) is highly sensitive to its hyperparameter configuration. To systematically identify an effective model structure, we initially employed a **Grid Search Cross-Validation (Grid-SearchCV)**. The search space was defined across four critical dimensions governing the ensemble’s learning dynamics and tree architecture (Table 3).

Hyperparameter	Search Space	Best Parameter
n_estimators	[500, 1000]	1000
learning_rate	[0.01, 0.05, 0.1]	0.05
max_depth	[3, 4, 5]	3
subsample	[0.8, 0.9]	0.9

Table 3. GridSearchCV parameter space and optimal configuration.

While GridSearchCV achieved a strong CV RMSE of 0.1264, it suffers from two critical limitations. First, its discrete nature creates ‘blind spots,’ risking failure to find the global optimum. Second, exhaustive search scales poorly; the high computational cost (180 fits for 36 configurations) severely constrains search boundary expansion.

To comprehensively address the “blind spot” and the combinatorial explosion of the discrete grid, we subsequently transitioned to **RandomizedSearchCV**. The true advantage of this stochastic optimization lies in its ability to sample

from continuous probability distributions rather than being confined to a finite list of predefined values (Table 4).

Hyperparameter	Search Range	Best Parameter
n_estimators	[300, 1500]	1452
learning_rate	[0.01, 0.10]	0.0249
max_depth	[3, 8]	3
subsample	[0.6, 1.0]	0.6795
min_samples_split	[2, 20]	10
min_samples_leaf	[1, 10]	7

Table 4. RandomizedSearchCV parameter distributions and the identified optimal configuration.

Beyond achieving higher predictive accuracy, RandomizedSearchCV demonstrated superior computational efficiency. By intelligently sampling from continuous distributions, it located a more optimal global minimum in just 100 iterations, compared to the 180 fits required by the exhaustive Grid Search.

Optimization Strategy	CV RMSE	Test RMSE	MPE
GBR (Grid Search)	0.1264	0.1353	14.49%
GBR (Random Search)	0.1249	0.1317	14.08%

Table 5. Performance comparison between Grid Search and Randomized Search optimization.

As demonstrated in Table 5, RandomizedSearchCV undeniably yields superior predictive performance.

4.6. Architectural Synthesis: From Hybrid to Stacking

While the Gradient Boosting Regressor (GBR) demonstrated strong individual performance, it suffers from a fundamental limitation inherent to tree-based models: the lack of **extrapolation capability**. Because decision trees partition the feature space into constant leaf values, they cannot inherently predict values beyond the maximum price encountered in the training set. To mitigate this, we initially implemented a **Hybrid Ensemble** by blending our regularized Lasso model with GBR. Using cross-validation, we performed a grid search across a range of blending ratios (from 1:0 to 0:1) to identify the optimal weight distribution between linear trends and non-linear nuances.

Transitioning to a Stacking Architecture, we evaluated three configurations: (Lasso + Ridge + GBR), (Lasso + GBR), and (Ridge + GBR). The (Ridge + GBR) ensemble yielded the highest validation error (CV RMSE: 0.1250), whereas the lean (Lasso + GBR) combination matched the performance of the full three-model stack (CV RMSE: 0.1232). This demonstrates that Lasso’s L1-driven feature selection provides a stronger, more complementary linear signal to

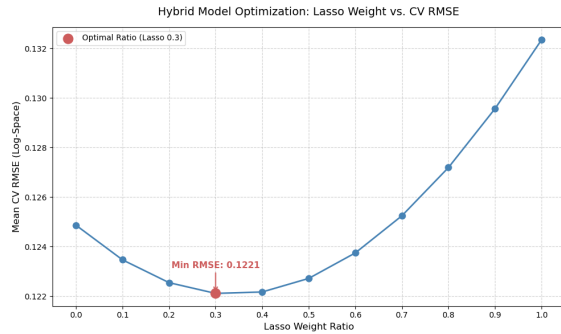


Figure 6. Cross-validation RMSE across different blending ratios for the Hybrid Ensemble.

the non-linear GBR than Ridge.

4.7. The Final Frontier: Integrating XGBoost

To further push the performance boundary, we integrated **XGBoost** into our ensemble. Unlike standard GBR, XGBoost incorporates advanced L1 and L2 regularization directly into its objective function, making it superior at handling the high-dimensional sparsity present in the Ames dataset. In our final architectural optimization, we pruned the redundant Ridge model and established a three-pillar Stacking framework comprising **Lasso, GBR, and XGBoost**. This synthesis successfully leveraged Lasso’s global anchoring with the dual tree-based models’ mastery of non-linear interactions, ultimately achieving our most robust and precise predictive performance (Test RMSE: 0.1287).

5. Contributions

Our project compares linear, regularized, and ensemble methods on the Kaggle House Prices dataset. We started from a plain linear regression and ended with a stacked ensemble of Lasso, Gradient Boosting, and XGBoost. We did not chase the lowest RMSE only. A lot of time went into figuring out why each model failed before we moved to the next one. The OLS baseline, for example, gave huge weights to sparse one-hot features. Once we understood that, the move to Lasso and Ridge made sense, and the same kind of reasoning led us to the tree-based models later on.

The full list of models we built and tuned is: OLS as a baseline; Lasso and Ridge with cross-validated regularization; Gradient Boosting with both grid search and randomized search; XGBoost; a weighted hybrid of Lasso and Gradient Boosting; and three stacking setups, with the final one being Lasso + Gradient Boosting + XGBoost. We also compared raw features, StandardScaler, and the Yeo–Johnson transform, and ran validation curves on six gradient boosting hyperparameters to see how stable the tuned configuration really was. A lot of the work in the Process section (the

Condition2.PosN outlier diagnosis, the leaf/split stress testing, etc.) does not show up in the final RMSE table but took significant time, so we want to flag it here.

For libraries, scikit-learn did most of the heavy lifting (cross-validation, hyperparameter search, the stacking and voting regressors). XGBoost was used for the boosting model, and pandas, NumPy, SciPy, Matplotlib, and Seaborn for the rest. Two public Kaggle notebooks (jesucristo, 2017; mviola, 2018) helped us with some of the feature-engineering choices.

6. AI Usage

We used AI tools (mostly Claude) during the project. How much we relied on them depended on what we were doing.

We did not use AI much for ideation. The general direction – comparing linear, regularized, and ensemble models on a tabular regression task – came from the course content and from looking at top Kaggle solutions. The choice of which models to add at which stage was ours. We sometimes asked AI for a quick second opinion (for example, whether stacking on top of a strong gradient boosting model was worth trying), but we did not let it pick the direction.

Coding was where AI helped the most. A lot of the scikit-learn setup is repetitive – KFold splits, GridSearchCV parameter grids, stacking pipelines, and matplotlib plots like the validation curves and feature importance charts. AI was a fast way to get the first version of those, which we then read through, ran, and tweaked to fit our pipeline. We also used it to figure out a few scikit-learn error messages and parameter combinations that did not behave as we expected.

For literature review we did not use AI at all. The references in this paper were found through the course materials, the Kaggle forum, and following citations from one paper to another, and we read them ourselves. We were aware of the issue with hallucinated citations, so we manually checked each reference against the actual source.

Sometimes we used AI as a quick reference when a concept was not clear, for example why `StandardScaler` alone is not enough when the features are heavily skewed, or how out-of-fold predictions feed into a stacking ensemble. These were starting points – the actual choices in our pipeline came from our own experiments.

For writing, AI was used to polish the language of paragraphs we had already drafted, and to fix awkward phrasing. The numbers, the figures, and the analysis itself are ours, and we take responsibility for the contents of this report.

References

- Breiman, L. Stacked regressions. *Machine Learning*, 24(1), 1996.
- Chen, T. and Guestrin, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016.
- Friedman, J. H. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001.
- Hoerl, A. E. and Kennard, R. W. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 1970.
- jesucristo. #1 house prices solution. <https://www.kaggle.com/code/jesucristo/1-house-prices-solution-top-1>, 2017.
- mviola. House prices eda, lasso & lightgbm [0.11635]. <https://www.kaggle.com/code/mviola/house-prices-eda-lasso-lightgbm-0-11635>, 2018.
- Tibshirani, R. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 1996.
- Wolpert, D. H. Stacked generalization. *Neural Networks*, 5(2), 1992.